

Airport Management System

Behavioral Design Patterns Documentation

Prepared by: Ajla Alcani & Eneida Balliu

Course: Software Modeling and Design

Date: May 14, 2026

Document Overview

This document presents the implementation and structural mapping of Behavioral Design Patterns within the Airport Management System (AMS). While previous models focused on dynamic state transitions and message sequencing, this document formally defines the static class architectures used to solve specific structural problems. It specifically details the Observer Pattern applied to bridge the Human Resources and System Administration modules.

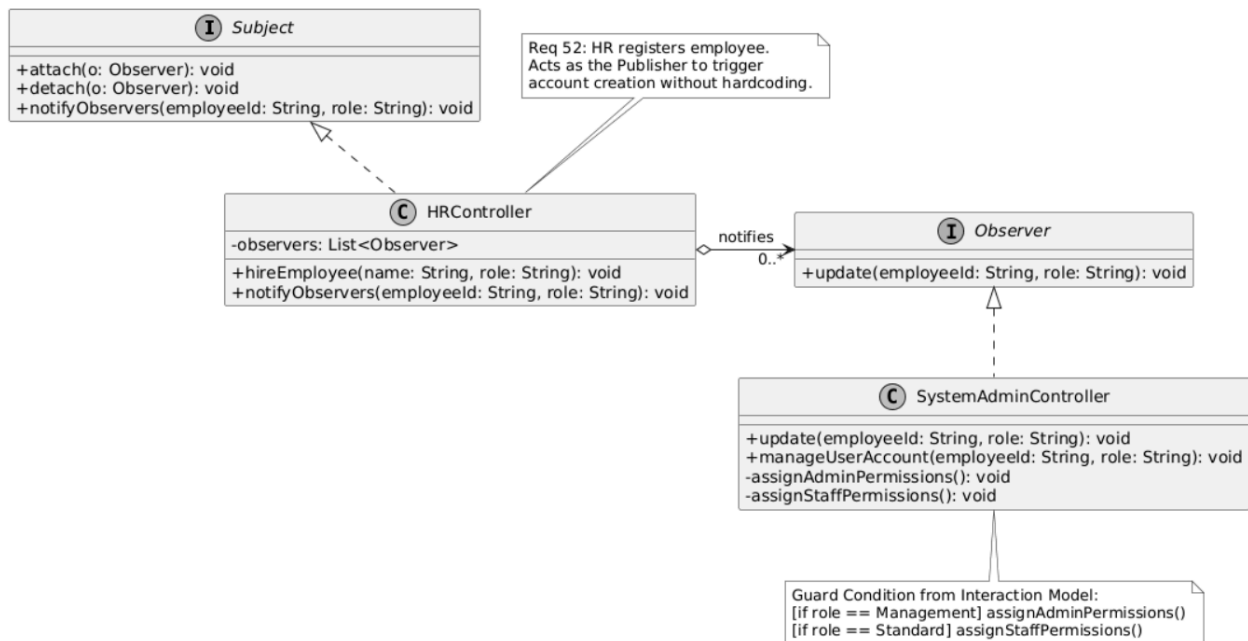
Objective

The primary goal of this document is to justify our architectural choices based on established software design principles. It provides strict UML Class Diagrams to demonstrate how the system fulfills its functional requirements while safely handling alternative flows and preventing tight coupling between independent subsystems.

Design Pattern 1: Observer Pattern (Module Integration: HR & System Admin)

- **Requirement Mapped:** Req 52 (HR registers employee records) and System Admin Account Creation.
- **Problem:** In our interaction models, when the Human Resources module (Ajla) registers a new employee, it needs to trigger the System Administration module (Eneida) to automatically generate a digital user account. If we hardcoded the System Admin classes directly into the HR code, the two modules would be tightly coupled, making the system fragile if administrative rules ever change.
- **Solution:** We applied the Behavioral **Observer Pattern**. The **HRController** acts as the Subject (or Publisher). When an employee is hired, it broadcasts a notification. The **SystemAdminController** acts as the Observer (or Subscriber), listening for that event to create the account.
- **Justification:** This pattern is ideal because it allows us to establish relations between objects at runtime. The HR backend does not need to know the internal implementation of the Admin backend. It also securely manages our conditional logic: when the Admin observer receives the notification, it uses our predefined guard conditions `[if role == Management]` or `[if role == Standard]` to assign the correct permissions.

2. The Class Diagram



Behavioral Design Patterns Documentation

Prepared by: Alvi Elezi

Course: Software Modeling and Design

Date: May 14, 2026

Document Overview

This document presents the implementation and structural mapping of Behavioral Design Patterns within the Airport Management System (AMS). While previous models focused on dynamic state transitions and message sequencing, this document formally defines the static class architectures used to solve specific structural problems. It specifically details the Facade Pattern used in the Flight Operations Module.

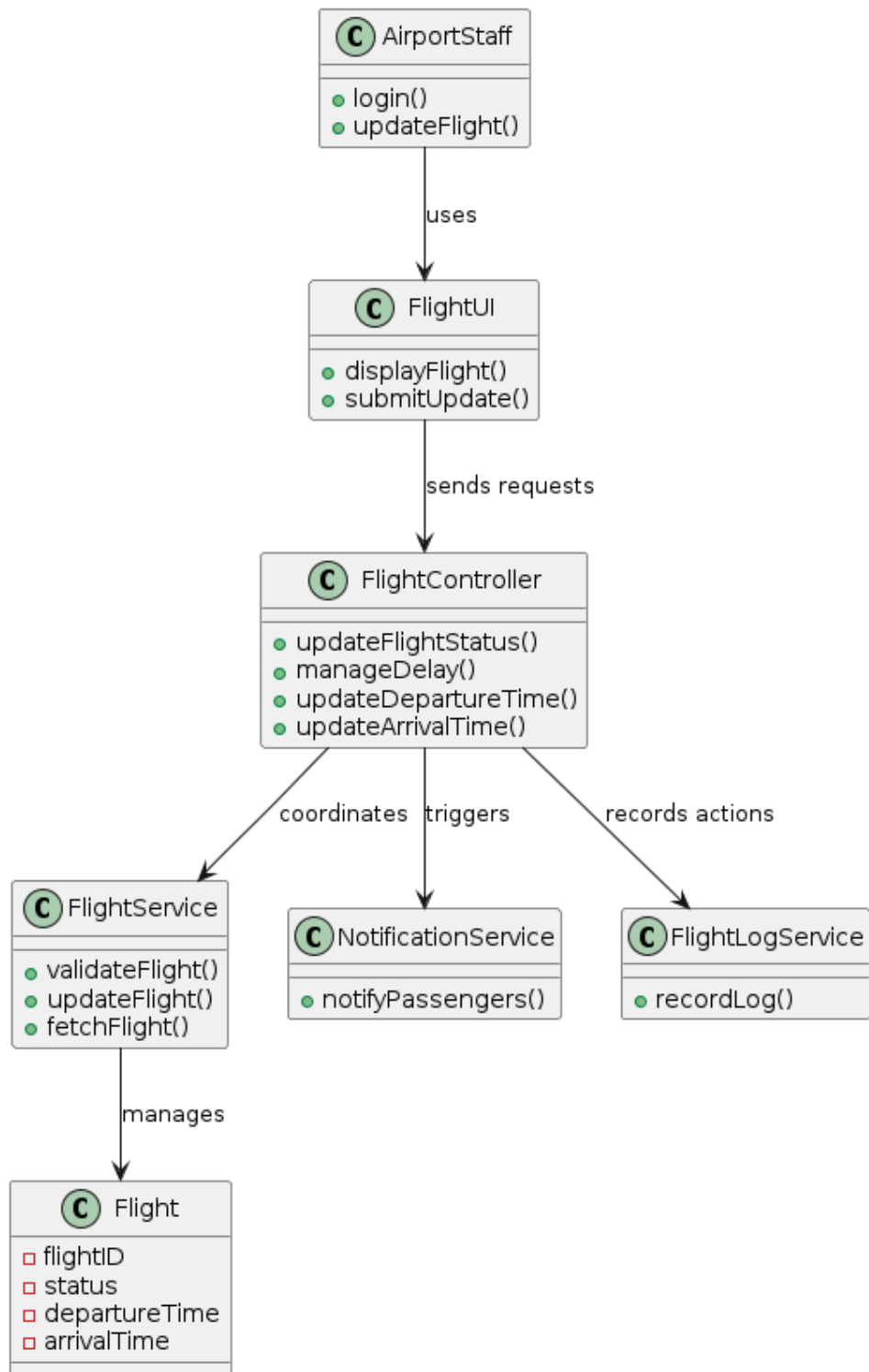
Objective

The primary goal of this document is to justify our architectural choices based on established software design principles. It provides strict UML Class Diagrams to demonstrate how the system fulfills its functional requirements while safely handling alternative flows and preventing tight coupling between independent subsystems.

Design Pattern 2: Facade Pattern (Module Integration: Airport Staff)

- **Requirements Mapped:**
 - Flight status management
 - Flight delay handling
 - Departure and arrival time updates
 - Passenger notification handling
 - Operational logging and database persistence
 - Centralized coordination of flight operations
- **Problem:** The Flight Operations Module contains multiple interconnected subsystems such as flight validation, scheduling, notifications, logging, and database management. If the user interface interacted directly with all these components, the system would become highly coupled, difficult to maintain, and more complex to extend. Any modification to one subsystem could require changes across several parts of the application.
- **Solution:** The system implements the **Facade Pattern** through the **FlightController** class. The controller acts as a centralized access point between the user interface and the internal operational subsystems. Instead of directly communicating with services such as **NotificationService** and **FlightLogService** the UI sends all operational requests to the **FlightController**, which coordinates the required actions internally.
- **Justification:** The Facade Pattern is appropriate for the Flight Operations Module because it simplifies communication with complex operational subsystems while reducing coupling between layers. The pattern improves maintainability by isolating subsystem changes from the user interface and centralizing operational workflows within the controller layer. It also enhances scalability, as additional operational services can be integrated behind the facade without requiring modifications to external components.

2. The Class Diagram



Airport Management System

Behavioral Design Patterns Documentation

Prepared by: Albina Blloshmi and Betül Kaan

Course: Software Modeling and Design

Date: May 14, 2026

Document Overview

This document presents the implementation and structural mapping of the Behavioral Chain of Responsibility Design Pattern within the Airport Management System (AMS). While previous system models focused on operational workflows and subsystem interactions, this document formally defines the request-handling architecture used to manage passenger service processes. It specifically details how passenger requests are sequentially processed through multiple handlers, including check-in, baggage registration, baggage tracking, and baggage claim services, without creating tight dependencies between system components.

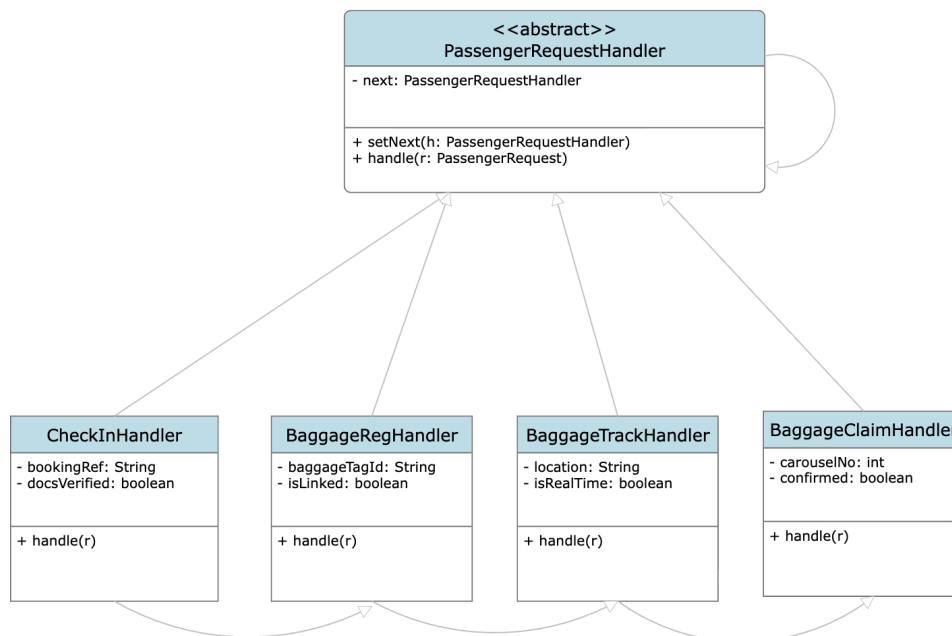
Objective

The primary goal of this document is to justify the architectural decisions behind the implementation of the Chain of Responsibility Pattern based on established software design principles. It provides UML Class Diagrams to demonstrate how the system efficiently processes passenger-related requests through independent handlers while ensuring flexibility, scalability, and maintainability. The approach prevents tight coupling between modules and supports future expansion of airport service functionalities.

Design Pattern 3: Chain of Responsibility Pattern (Passenger Request Handling)

- **Requirement Mapped:** Passenger service request processing within the Airport Management System, including check-in, baggage registration, baggage tracking, and baggage claim handling.
- **Problem:** In our Airport Management System, passenger requests can involve multiple services depending on the situation. For example, a passenger may first require check-in validation, then baggage registration, baggage tracking, or baggage claim confirmation. If all these responsibilities were hardcoded into one centralized controller, the system would become tightly coupled and difficult to maintain. Any modification in one process (e.g., baggage tracking rules) could negatively affect the entire request-handling workflow.
- **Solution:** We applied the Behavioral Chain of Responsibility Pattern. The abstract PassengerRequestHandler acts as the base handler and defines the request-processing chain through the setNext() and handle() methods. Each concrete handler (CheckInHandler, BaggageRegHandler, BaggageTrackHandler, and BaggageClaimHandler) is responsible for processing a specific passenger-related request. If a handler cannot fully process the request, it forwards it to the next handler in the sequence. This approach enables requests to move dynamically through the chain without hardcoding dependencies between modules.
- **Justification:** This pattern is ideal because it promotes loose coupling between request senders and handlers. Each class has a single responsibility, making the system easier to extend and maintain. For example, new passenger service handlers can be added without changing the existing logic. Additionally, the sequential handling process ensures that airport operations such as check-in, baggage registration, tracking, and claim management are processed in an organized and scalable manner.

2. The Class Diagram



Behavioral Design Patterns Documentation

Prepared by: Ani Velo

Course: Software Modeling and Design

Date: May 14, 2026

Document Overview

This document presents the implementation and structural mapping of Behavioral Design Patterns within the **Security Management Module** of the Airport Management System (AMS). While previous models focused on dynamic message sequencing, this document formally defines the static class architectures used to solve specific security and validation problems. It specifically details the **Proxy Pattern** as they apply to passenger screening and emergency protocols.

Objective

The primary goal of this document is to justify our architectural choices based on established software design principles. It provides strict UML Class Diagrams to demonstrate how the security module fulfills its functional requirements while maintaining "thin" user interfaces and automated audit compliance.

Design Pattern 1: Proxy Pattern (Divergent Checkpoint Verification Pipeline)

Requirements Mapped (Group 1):

- UC_SEC_01: Passenger Security Screening
- UC_SEC_02: Restricted Area Access Control
- UC_SEC_05: Prohibited Item Logging

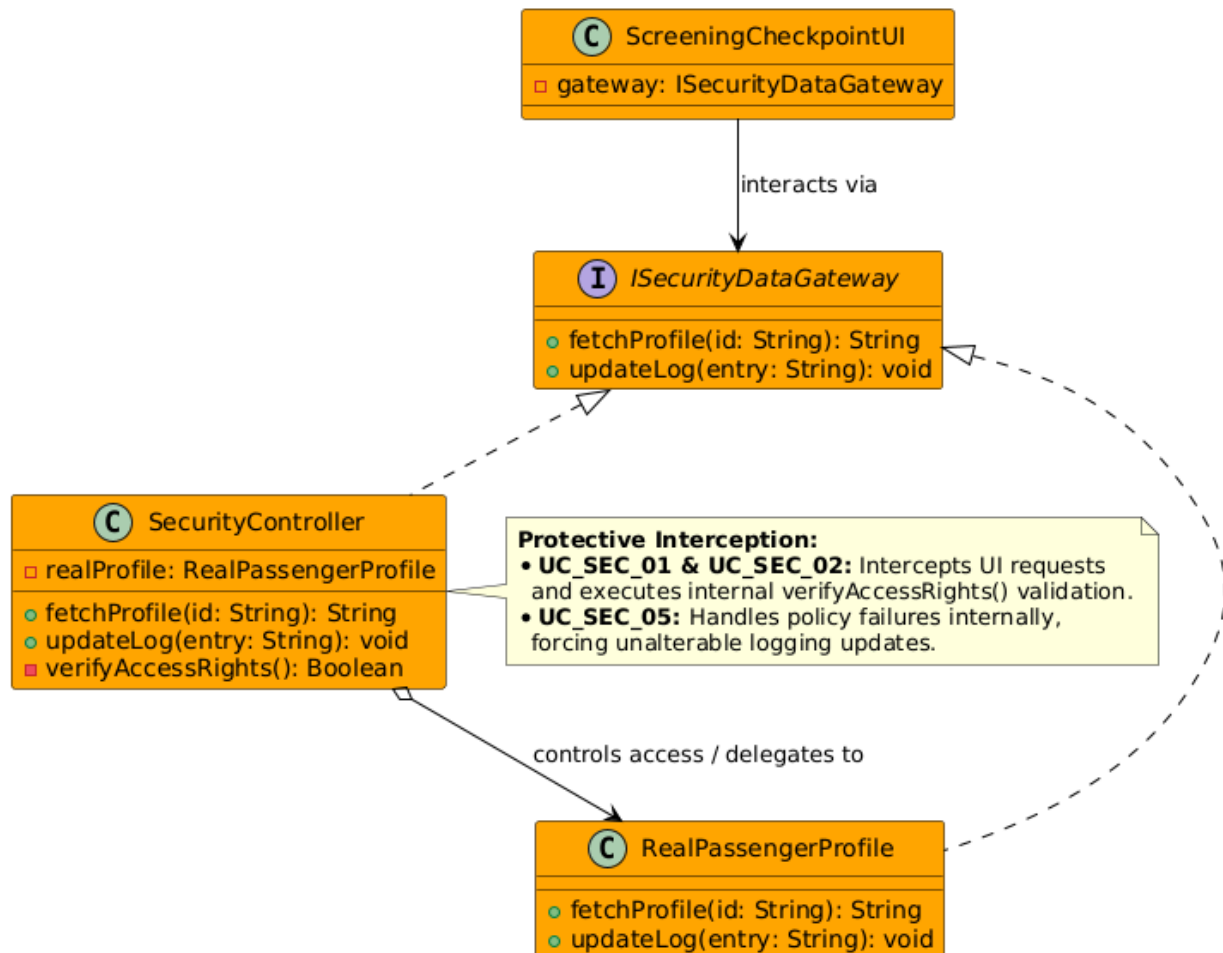
Problem: During passenger security screening (UC_SEC_01) and access authorization checks (UC_SEC_02), system operators and checkpoint interfaces need to fetch highly sensitive passenger profile data and log system-level access events. Directly allowing a user interface component (**ScreeningCheckpointUI**) to call, mutate, or read data from the underlying data entities (**PassengerProfile**) introduces severe security vulnerabilities, data misuse risks, and tight architectural coupling. If the interface layer is compromised, the data persistence tier is completely exposed.

Solution: We applied the Structural **Proxy Pattern**. We established a common interface wrapper contract, **ISecurityDataGateway**, which exposes operations like **fetchProfile(id: String)** and **updateLog(entry: String)**. The actual database connection logic is isolated inside a concrete **RealPassengerProfile** class. We then introduce the **SecurityController** to act as the protective **Proxy** object implementing that same interface. The **ScreeningCheckpointUI** only interacts with the proxy controller. When a method is invoked, the proxy executes an internal **verifyAccessRights()** interceptor routine. It only forwards the request to the real profile object if security criteria are satisfied.

Justification: This pattern creates a controlled boundary around your sensitive persistence layer data. It allows the system to seamlessly perform logging audits, parameter sanitization, and access token validation without complicating the core business logic of the real entity data classes. If a screening reveals a weapon or illicit substance (UC_SEC_05), the proxy controller intercepts the failure, automatically flags the profile status, and routes a protected transaction write to initialize a contraband entry, ensuring no unauthenticated interface can manipulate or erase active security violation registries.

2. The Class Diagram

Security Management Module - Proxy Pattern Class Architecture (Group 1)



Airport Management System

Behavioral Design Patterns Documentation

Prepared by: Ameo Lulaj

Course: Software Modeling and Design

Date: May 14, 2026

Document Overview

This document presents the implementation and structural mapping of Behavioral Design Patterns within the Airport Management System (AMS). While previous models focused on dynamic state transitions and message sequencing, this document formally defines the static class architecture used to solve specific behavioral problems. It specifically details the State Pattern applied to manage the lifecycle of the shared airport resources handled by the Resource Management module - gates, runways, staff and equipment.

Objective

The primary goal of this document is to justify our architectural choices based on established software design principles. It provides strict UML Class Diagrams to demonstrate how the system fulfills its functional requirements while safely handling alternative flows (conflicts, faults, unavailability) and preventing tight coupling between a resource and its many lifecycle behaviours.

State Pattern (Behavioral)

Intent: Allow a resource object to alter its behaviour when its internal state changes, so the object appears to change its class.

Problem in AMS: Every shared resource moves through a defined lifecycle - Available, Reserved, Assigned, In Use, Under Maintenance and Unavailable. Implementing this with large if/switch blocks on a single status flag scatters the transition rules, makes invalid transitions easy to introduce, and violates the Open/Closed Principle whenever a new state is required.

Solution: The Resource (context) delegates all behaviour to a ResourceState object. Each concrete state class (AvailableState, ReservedState, AssignedState, InUseState, UnderMaintenanceState, UnavailableState) implements only the transitions that are legal from that state and rejects the rest, then swaps the context to the next state. This is the class-level realisation of the UC-ARM state machine diagrams.

SOLID justification: OCP - a new lifecycle state is added by writing one new state class, with no change to existing states or to the Resource. SRP - each state class is solely responsible for the rules of exactly one lifecycle stage.

Mapped requirements: UC-ARM-01 (Gate Allocation), UC-ARM-02 (Runway Scheduling), UC-ARM-03 (Staff Allocation), UC-ARM-04 (Equipment Management) - directly implements their state diagrams.

State Pattern - Resource Lifecycle

